

# 低精度（FP8/FP4）架构分析报告

以昇腾 910B 真机实验回答四个架构问题，并探讨下一代设计方向

载体模型：ET-BERT（加密流量识别） · 平台：昇腾 910B4 · 日期：2026-06-05

## 摘要

一句话结论：在 910B 上实测，ET-BERT 的端到端时延由两类开销主导——① **host/kernel 运行时调度开销**，与② **访存（片上数据搬运，主要是把权重/激活填入 L1 的 MTE2 通路）**；计算（CUBE/mac）并非瓶颈。host 开销属运行时/编译层问题，随整图下沉与算子融合会被持续优化；一旦 **host 被优化掉**，访存即成为设备侧持久的核心瓶颈。

对加密流量识别这类负载（序列长度低、batch 小、模型矩阵维度小），算术强度天然偏低，**访存搬运**是设备侧的核心约束。低精度（FP8/FP4）的主要价值，不在于 4× 算力（该工况难以吃满），而在于**减小数据足迹、提升 cache 命中、降低访存压力**——这与访存优化方向一致、相互增益。

## 一、实验平台、方法与口径

**被测芯片（910B4）**：24 个 AI Core，每核含 1 个 CUBE 矩阵单元（16×16×16 MAC）与 2 个向量单元（AIV）；存储层级 L0 → L1（512KB/核）→ L2（96MB 共享）→ HBM（392 GB/s）；峰值 FP16 280 TFLOPS、INT8 560 TOPS。

**AI Core 内部按 pipe 计时（后文反复用到）**：mac=CUBE 计算；mte1=L1↔L0 搬运；**mte2=HBM/L2→L1 填充（访存搬运）**；fixpipe=L0C→输出 / 量化 requant。

**精度类比**：FP8/FP4 无对应硅，用 910B 的 FP16/INT8 做数据密度类比（INT8≐FP8 1 字节、INT4≐FP4 0.5 字节），算力倍数按下一代假设外推。

**测量手段（均为厂商原生工具，独立于本项目模型，可复现）**：ais\_bench 测端到端时延；msprof 的 ArithmeticUtilization / PipeUtilization / L2Cache / Memory 采集 CUBE 利用率、各 pipe 耗时、cache 命中率与各级带宽；AMCT 做 INT8 训练后量化（PTQ）后实测。

**口径说明**：本文“CUBE 利用率/MFU”指 CUBE 实际做有用 MAC 的时间占比（msprof aic\_mac\_\*\_ratio 按算子耗时加权）；“访存”按层级区分为“HBM 外部带宽”与“片上 L2→L1 搬运带宽”，二者分开讨论。

## 二、瓶颈定位：host 调度与访存搬运

### 2.1 时延分解：固定开销占主导

实测 ET-BERT (FP16) 端到端时延随 batch 变化，做线性拟合可分离“固定开销”与“边际计算”：

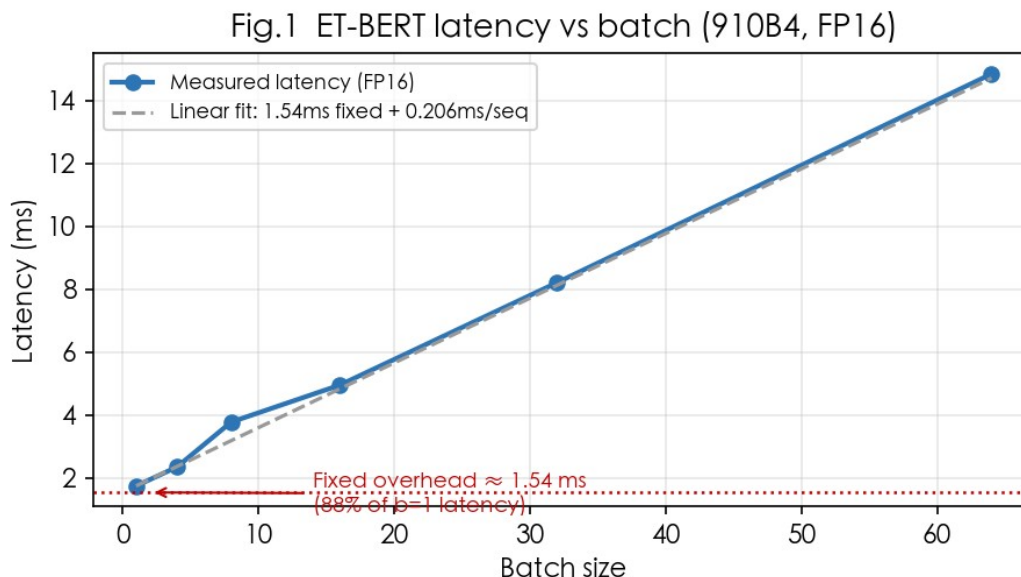


图1 时延随 batch 变化：固定开销约 1.55ms，约占  $b=1$  时延的 88%

边际计算仅 0.21 ms/样本，固定开销约 1.55 ms。即处理单样本时约 88% 的时延与计算量无关，来自约 330 个 kernel 的 host 派发与众多小 kernel 的启停延迟。该固定开销属运行时层面，可通过整图下沉、算子融合、攒批持续优化（见第五节）。

### 2.2 设备侧主导 pipe 是访存（mte2），而非计算

把 host 开销剥离后，设备侧关键路径由各 pipe 的最大值决定。实测各 batch 下 AIC 内部主导 pipe 恒为 mte2（把数据填入 L1 的访存搬运），其耗时持续高于 mac（CUBE 计算）：

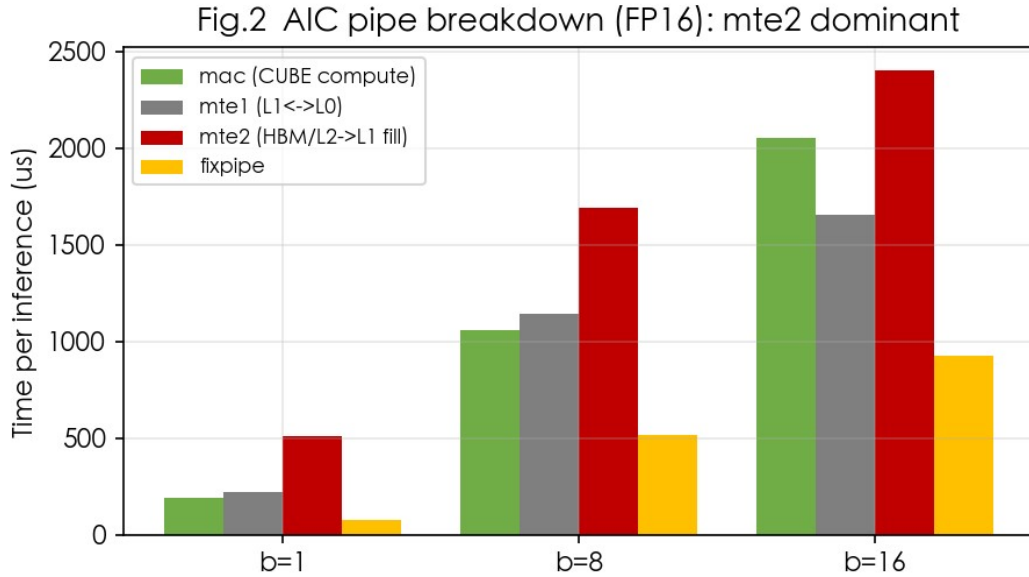


图2 AIC pipe 分解 (FP16) : mte2 (红, 访存填充) 在各 batch 均为最大项

用 mac/mte2 比值刻画设备侧的 roofline 位置：该比值在所测全部工况 ( $S=128/256/512 \times \text{batch}=1/10$ ) 始终小于 1 (**0.37–0.89**)，主导 pipe 恒为 mte2 —— 设备侧在整个加密流量识别工况区间处于访存搬运受限一侧。

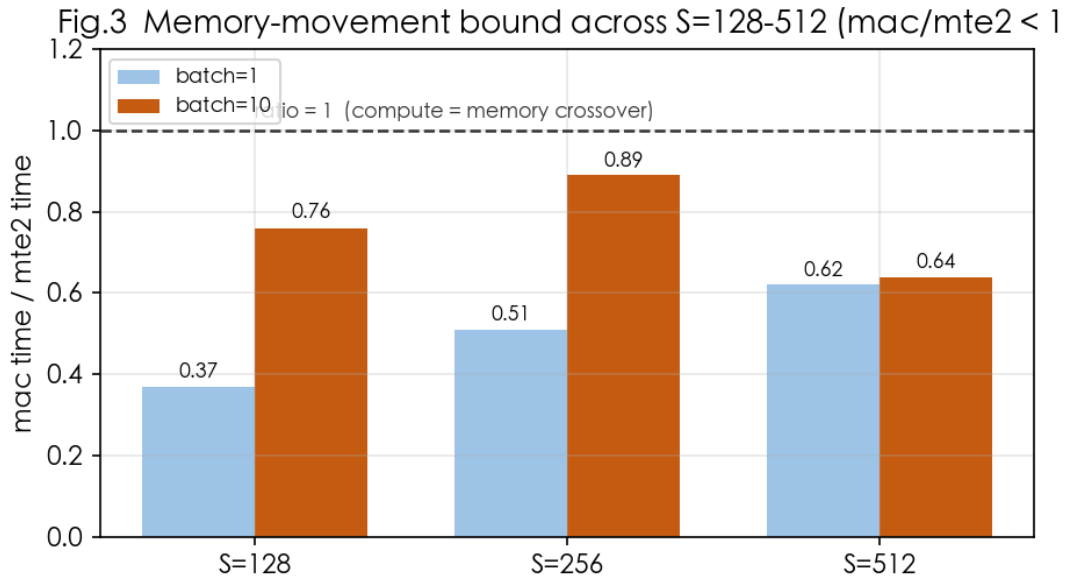


图3  $S=128/256/512 \times b=1/10$  全工况 mac/mte2 < 1：设备侧处于访存受限侧

结合工况分析：加密流量识别的序列长度 (256/512) 与 batch (约 10) 都较小，模型矩阵维度也小，单位权重加载所摊到的计算行数有限 → 算术强度低 → 设备侧天然偏访存搬运受限。S=512 b=10 时注意力  $O(S^2)$  激活搬运使 mte2 升至 7460 $\mu$ s (mac 4741 $\mu$ s)，mac/mte2 = 0.64，访存主导更显著。

## 2.3 两层访存：外部 HBM 与片上 L2→L1

“访存受限”需区分两个层级。实测 cache 命中率与带宽显示：权重/激活的反复取用大部分被片上 L2/L1 接住，外部 HBM 在稳态下利用率很低；真正在设备侧构成约束的是片上 L2→L1 的填充 (mte2)。

Fig.5 Cache absorbs reuse; external HBM near idle

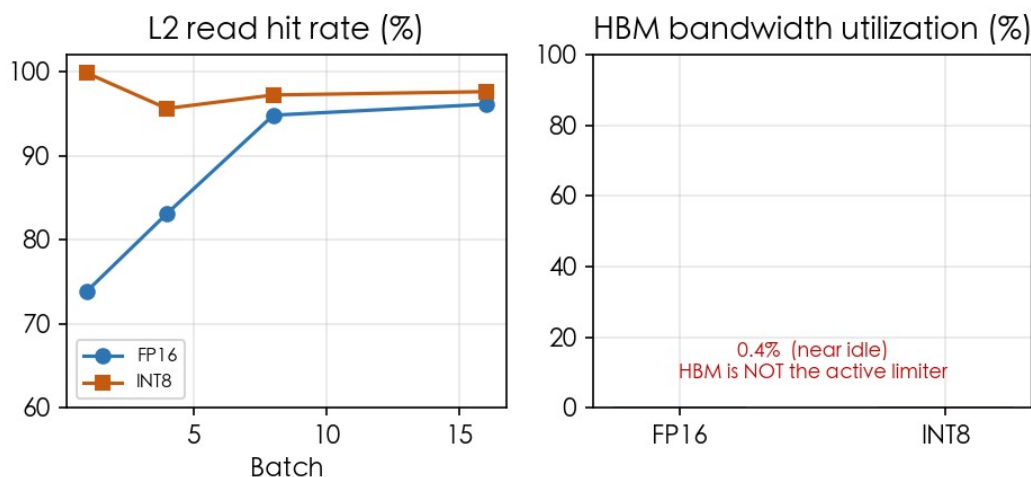


图5 片上 L2 读命中率高 (74~99%)，外部 HBM 带宽利用率仅约 0.4%

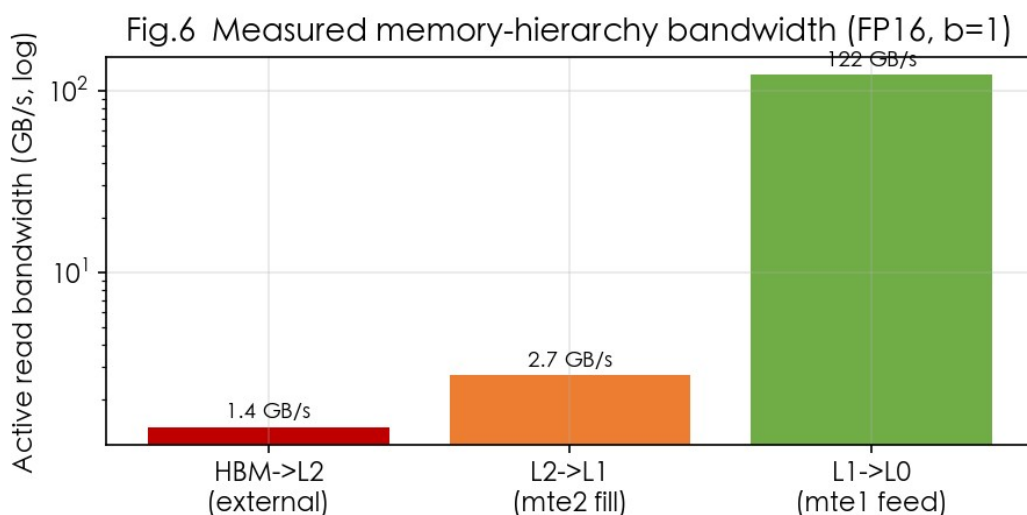


图6 实测三级带宽：外部 HBM 1.4 / 片上 L2→L1 2.7 / L1→L0 122 GB/s (对数轴)

**要点：**外部 HBM 带宽在当前工况未成为限制因素（被 96MB L2 吸收）；但片上 L2→L1 填充带宽是设备侧关键路径。两者都属“访存”，架构上需分别看待——前者决定“能否把工作集留在片上”，后者决定“留下后填得多快”。

三、四个问题的详细回答

Q1 混合精度下 MFU 如何公平计算？

结论：混合精度没有统一峰值，需对每个算子按其自身精度的峰值分别归一、再按算子耗时加权。同一负载存在两种合理 MFU 口径（CUBE 内与端到端），二者各自有意义但分母不同，跨精度比较时需先声明口径再比较。

公式（按 msprof 定义）：

单算子的精度专属利用率（msprof 字段 aic\_mac\_\*\_ratio）：

$$u\_op^{(fp16)} = \frac{\text{该算子实际完成的 FP16 MAC 数}}{\text{该算子 aicore\_time 内 FP16 峰值能完成的 MAC 数}}$$

$$u\_op^{(int8)} = \frac{\text{该算子实际完成的 INT8 MAC 数}}{\text{该算子 aicore\_time 内 INT8 峰值能完成的 MAC 数}}$$

纯 FP16 算子  $u^{(fp16)} > 0$ 、 $u^{(int8)} = 0$ ；纯 INT8 算子反之；混精度算子（如 FP16 输入×INT8 权重）二者可同时为正。

整网 CUBE 内 MFU（按 aicore\_time 加权）：

$$MFU\_cube^{(fp16)} = \frac{\sum op \ t\_op \cdot u\_op^{(fp16)}}{\sum op \ t\_op}$$

$$MFU\_cube^{(int8)} = \frac{\sum op \ t\_op \cdot u\_op^{(int8)}}{\sum op \ t\_op}$$

$$MFU\_cube\_total = MFU\_cube^{(fp16)} + MFU\_cube^{(int8)}$$
 (无需统一峰值)

端到端 MFU（以 wall\_clock 为分母，反映系统级 CUBE 占用率）：

$$MFU\_e2e = \frac{mac\_time}{wall\_clock}$$
 (mac\_time = AIC mac pipe 累计耗时)

符号表：

| 符号               | 含义                          | 数据来源                             |
|------------------|-----------------------------|----------------------------------|
| t_op             | 单算子 aicore_time (μs)        | msprof op_summary                |
| u_op^(fp16/int8) | 算子相对各自精度峰值的 CUBE 利用率        | msprof aic_mac_fp16/int8_ratio   |
| mac_time         | 整网 AIC mac pipe 时间 (μs/inf) | msprof PipeUtilization 累加 ÷ loop |
| wall_clock       | 端到端推理时延 (μs/inf)            | ais_bench 干净时延                   |
| FP16/INT8 峰值     | 280 TFLOPS / 560 TOPS       | 910B4 规格（一次 MAC 计两次 FLOP）        |

用真机数据走一遍（ET-BERT，FP16-only 路径）：

| 工况        | mac_time (μs/inf) | wall_clock (μs) | MFU_cube (msprof) | MFU_e2e = mac/wall |
|-----------|-------------------|-----------------|-------------------|--------------------|
| FP16 b=1  | 189               | 1758            | 32.3%             | 10.7%              |
| FP16 b=16 | 2055              | 4944            | 74.5%             | 41.6%              |
| INT8 b=1  | 84                | 2208            | 14.3%             | 3.8%               |

| 工况        | mac_time (μs/inf) | wall_clock (μs) | MFU_cube (msprof) | MFU_e2e = mac/wall |
|-----------|-------------------|-----------------|-------------------|--------------------|
| INT8 b=16 | 892               | 5353            | 45.5%             | 16.7%              |

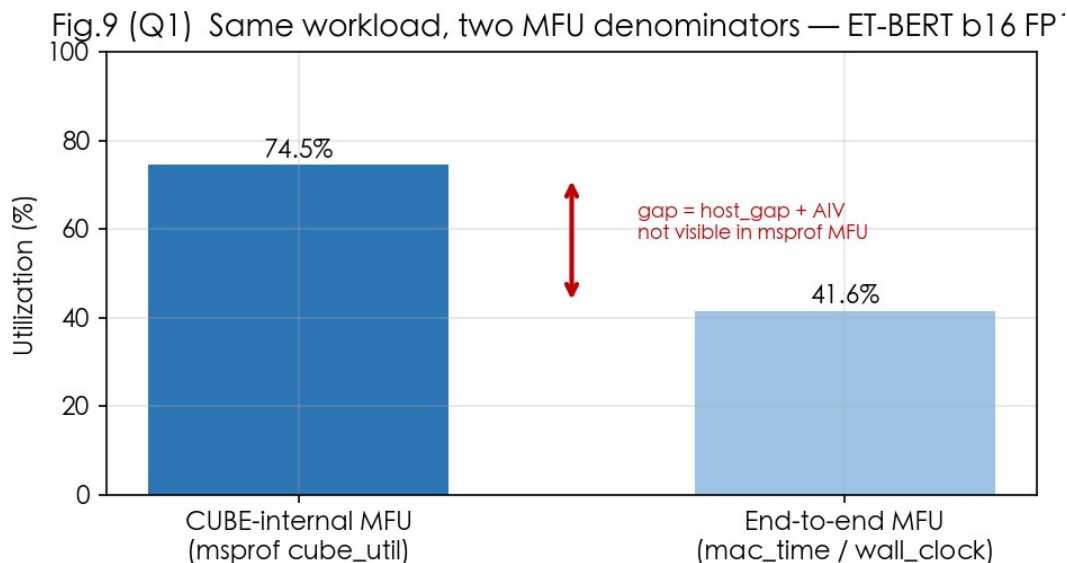


图9 同一工况 (ET-BERT b=16 FP16) 的两种 MFU: 差额 = host\_gap + AIV, msprof 口径不可见

是否符合直觉? 部分符合、部分需解释: (1) 同精度内 batch 越大 MFU 越高 — 符合直觉 (每 kernel 摊到更多 MAC, CUBE 更少空等)。 (2) INT8 的 MFU 数值低于 FP16 (45.5% < 74.5%) — 看似 INT8 更差, 但 INT8 峰值是 FP16 的 2×, 分母不同; 按 msprof 口径这正说明 INT8 算得太快、CUBE 更早空闲、相对自身峰值的利用率反而下降。 (3) INT8 的 MFU\_cube=45.5% 高于 INT8 的 MFU\_e2e=16.7% — 这 28.8 个百分点的差额, 是 host 调度 + AIV (含 requant 算子) 占用 wall\_clock 但不计入 aicore\_time 的部分。这是 msprof 口径看不见、需要端到端口径才暴露的系统损失 (见 Q3)。

使用建议 (两种口径各自的用途): MFU\_cube (msprof) 回答“当 CUBE 启用时它本身是否高效”, 适合跨精度比较 CUBE 单元的自身效率 (host 与 AIV 不进入分母); MFU\_e2e 回答“整段 wall\_clock 里 CUBE 真正在算的占比是多少”, 适合横向评估系统设计。跨精度直接比较 MFU\_cube 数值时 (如 45.5% vs 74.5%), 切勿据此结论“INT8 更差”——应该比较有效吞吐或端到端时延: 本次 INT8 b=16 实际更慢 26% (5.35 vs 4.93ms), 是 MFU\_cube 数字遮蔽的真实情况, 由 MFU\_e2e 与时延揭示。

## Q2 800T FP4 与 200T FP8 的访存带宽需求是否相同?

结论: 不相同。需区分搬运字节量 (每秒搬多少 MB) 与维持峰值所需带宽 (GB/s) 两个口径。用 roofline 图 (图 10) 直观看: 算力 ceiling 上移、HBM 斜线不变时, ridge 点 (compute-bound 与 memory-bound 的交界处) 右移, 更高 AI 才能维持 compute-bound。

offline: 当前 392 GB/s 让各精度都停在 ● 点; ★ 为各精度 compute-bound 临界, 向上箭

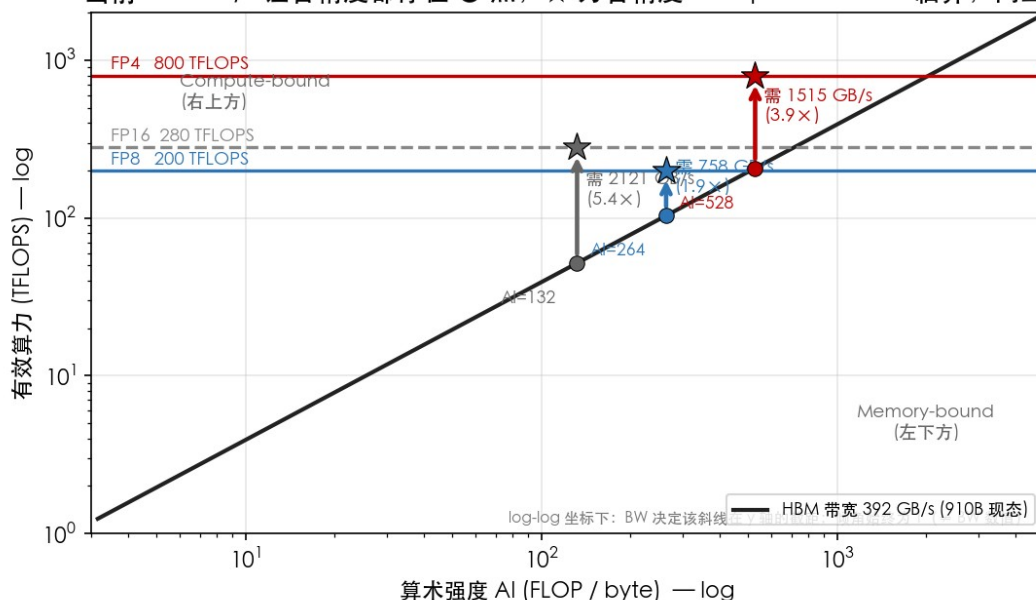


图10 Roofline 示意: 算力顶上移、HBM 斜线不变 → ridge 右移; 模型 AI 不变则更难达到 compute-bound

**Roofline 怎么读:** (i) 斜线 = HBM 带宽 × (算术强度 AI) —— 数据来多快、最多能跑多快; (ii) 水平顶 = 峰值算力 —— 计算单元自身上限; (iii) 二者相交处 AI<sub>ridge</sub> = peak / BW 是 compute-bound 与 memory-bound 的分界; (iv) AI 高于 ridge → compute-bound (卡算力); AI 低于 ridge → memory-bound (卡带宽)。

代入 910B HBM 带宽 392 GB/s 计算 ridge:

| 精度档       | 峰值算力       | AI <sub>ridge</sub> = peak ÷ BW | 说明                    |
|-----------|------------|---------------------------------|-----------------------|
| FP16 280T | 280 TFLOPS | 714 FLOP/byte                   | 910B 现状               |
| FP8 200T  | 200 TFLOPS | 510 FLOP/byte                   | 下一代设想 (算力较 FP16 反而略低) |
| FP4 800T  | 800 TFLOPS | 2041 FLOP/byte                  | 下一代设想 (算力 4×)         |

**两类带宽需求量化 (按 ridge 反推维持 compute-bound 的最低 BW):** 令 ridge ≤ ET-BERT 在该精度档下的 AI (FP8 时 ≈ 264、FP4 时 ≈ 528), 即 BW<sub>opt</sub> = peak / AI: FP8 (200T, AI=264) → BW<sub>opt</sub> ≈ 758 GB/s; FP4 (800T, AI=528) → BW<sub>opt</sub> ≈ 1515 GB/s。二者相差正好 2× —— 对应 Q2 公式 BW 需求 ∝ 算力÷密度。

以 ET-Bert 为例, 在不增加带宽的前提下, 单纯提高算力 (降低精度), 无法带来性能明显的提升, 因为其卡在带宽瓶颈区间。如果不考虑 host 等其他因素, 单纯只考虑 Roofline 模型, 下一代精度和算力演进后, 最佳的带宽分别是?

**回答 (基于 ET-BERT 实测 AI + roofline 模型推演, 对应图 10 中的 ★ 标记):** 把 ridge 对齐到模型的 AI 即可让该精度恰处于 compute-bound 临界点; 再往上加带宽不再有收益。代入 ET-BERT b=1 的 AI: FP16 路径 AI ≈ 132 (22.4 G ops / 170 MB FP16 权重), FP8 路径 AI ≈ 264 (位宽减半), FP4 路径 AI ≈ 528 (位宽四分之一) :



| 精度档       | 峰值         | ET-BERT AI    | BW_opt = peak / AI | 相对 910B 现态 (392 GB/s) |
|-----------|------------|---------------|--------------------|-----------------------|
| FP16 280T | 280 TFLOPS | 132 FLOP/byte | ≈ 2.12 TB/s        | 5.4×                  |
| FP8 200T  | 200 TFLOPS | 264 FLOP/byte | ≈ 758 GB/s         | 1.9×                  |
| FP4 800T  | 800 TFLOPS | 528 FLOP/byte | ≈ 1.52 TB/s        | 3.9×                  |

由该表可读出的三条事实（仅 **roofline** 视角，不含 **host** 与片上 **mte2**）：（1）**910B 现态 FP16** 已显著带宽不足：要让 FP16 在 ET-BERT 上达到 compute-bound 需 ~2.1 TB/s，约现态的 5×。（2）**FP8 200T** 是“带宽友好”的下一代选项：只需 ~1.9× 现态带宽（758 GB/s），近期工艺可达；但峰值算力较 FP16 反而下降。（3）**FP4 800T** 需 ~3.9× 带宽（1.5 TB/s）才能兑现 4× 算力红利；否则模型仍处于 memory-bound、算力被浪费。

提示：上表为 **roofline** 上界——它不包含 **host** 调度、片上 **mte2** 等 **wall-clock** 损失；端到端实际收益还需叠加这些路径的优化。

一般判据：BW 需求 ∝ 峰值算力 × 数据位宽；算力提升倍数 ÷ 密度提升倍数 即带宽需求的相对变化率。对照 910B FP16→INT8（算力 2× / 密度 2× = 抵消）→ BW 需求不变；下一代 FP8→FP4（算力 4× / 密度 2× = 翻倍）→ BW 需求翻倍。这就是 **roofline** 图里 FP4 ceiling 上移 4× 而 ridge 右移 4× 的几何含义。

Q3 FP8→FP4，MFU 会下降、需要更大带宽吗？

结论：MFU 会下降，并需要更大的访存带宽以让 compute 与 memory 重新匹配。实测在 910B 上将精度从 FP16 降到 INT8，端到端时延由 1.76 ms 升至 2.21 ms（增加约 26%）。

实验数据：同 batch 下 INT8 的 CUBE 利用率低于 FP16（b=16：46% vs 75%）；INT8 的 mac 时间确为 FP16 的约一半（2× 吞吐属实）；但端到端时延反而上升。INT8 端到端的 19–35% 落在量化/反量化算子（AscendQuant 与融合 dequant）上，由 **fixpipe/AIV** 承担。

Fig.11 (Q3) Why MFU drops: peak rises 4x but density only 2x (FP16→FP4)

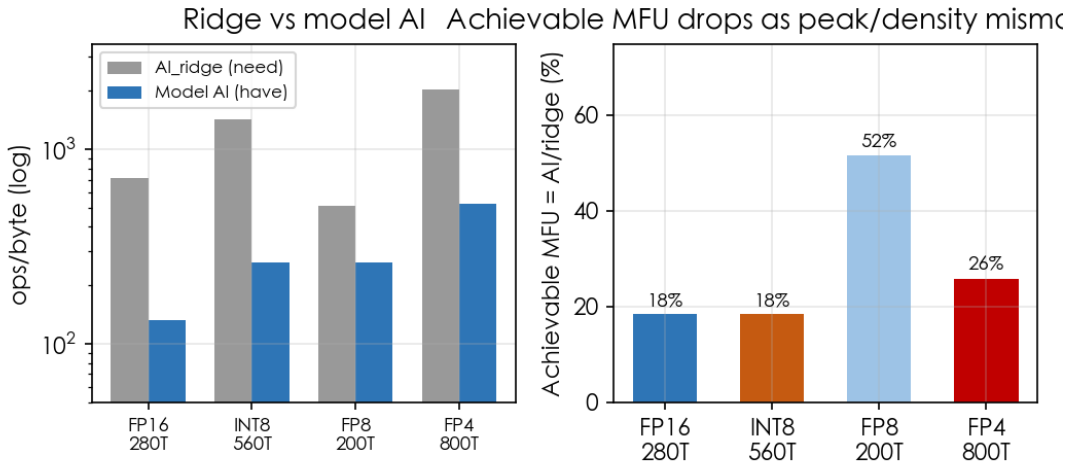


图11 (Q3) MFU 下降机理：算力 4× / 密度 2×，ridge 右移使可达 MFU 降低；同 AI 在更高 peak 下显示为更低利用率

把抽象原因落到图与公式上：

原因 1（“**mac** 缩短被关键路径吸收”——图 2/图 11 联读）：端到端 wall\_clock = max(mac, mte2, host\_gap, AIV) + 串行损失。实测 b=1 时 mac=189μs，但 mte2=508μs、host\_gap=13.7ms 才是关键



路径。将 mac 减半 (FP16→INT8) 后, 关键路径仍由 **mte2 + host\_gap** 决定, 时延几乎不动; 从 MFU 视角看, 分母 wall\_clock 不缩短而 mac 缩短 → 端到端 CUBE 占用率 ( $MFU_{e2e} = mac/wall$ ) 下降。

**原因 2 (“roofline 左移 → 算力被访存饿着”——图 10 / 图 11 联读)**: Q2 已说明: 精度档下移时, 峰值算力升、HBM 带宽不变 → **ridge 右移 (compute-bound 区域被压缩, memory-bound 区域扩大)**。模型自身的 AI 虽随精度提升 (FP16 132 → FP4 528), 但 ridge 上升更快 (FP16 ridge 714 → FP4 ridge 2041, 3× 倍率) —— 于是 AI 与 ridge 的相对比例 AI/ridge 反而下降 (FP16 0.185 → FP4 0.26 看似上升, 但叠加 4× peak 后实际  $MFU = AI/ridge \times peak\_ratio$  仍下降)。图 11 右栏给出三档可达 MFU 对比: FP4 800T 的可达 MFU 反而低于 FP8 200T, 即算力提升超出了密度提升能补偿的范围。

**不是 tile 量化损失 (澄清)**: CUBE 的原生分块为 fp16 16×16、int8 16×32、fp4 16×64; 只要矩阵的 M/N/K 均为对应分块的整数倍, 便无 tile 边缘浪费。ET-BERT 的 M=128, N=768/3072, K=768/3072/64/128 均为 64 的倍数 → 在 FP16/INT8/FP4 下 tile 量化效率均为 1, 即 **MFU 下降不来自 tile 损失, 而来自 roofline 几何关系**。

## Q4 16MB Cache、带宽、命中率、精度与对齐

**结论**: 16MB 无法容纳整模型 (OM FP16 约 165MB、INT8 约 85MB), 但 transformer 推理的复用模式不要求整模型常驻——只需当前层的权重工作集 (FP16 约 14MB、INT8 约 7MB) 能驻片即可。实测 L2 读命中率 FP16 74–96%、INT8 95–99.8%; 外部 HBM 带宽利用率约 0.4% —— 复用确实被 L2 吸收。

### Q4.1 神经网络层间的 Cache 复用模式 (系统结构视角)

Transformer 推理的访存模式可类比为**逐层流式 + 层内强复用**, 对体系结构读者其访存图景如下:

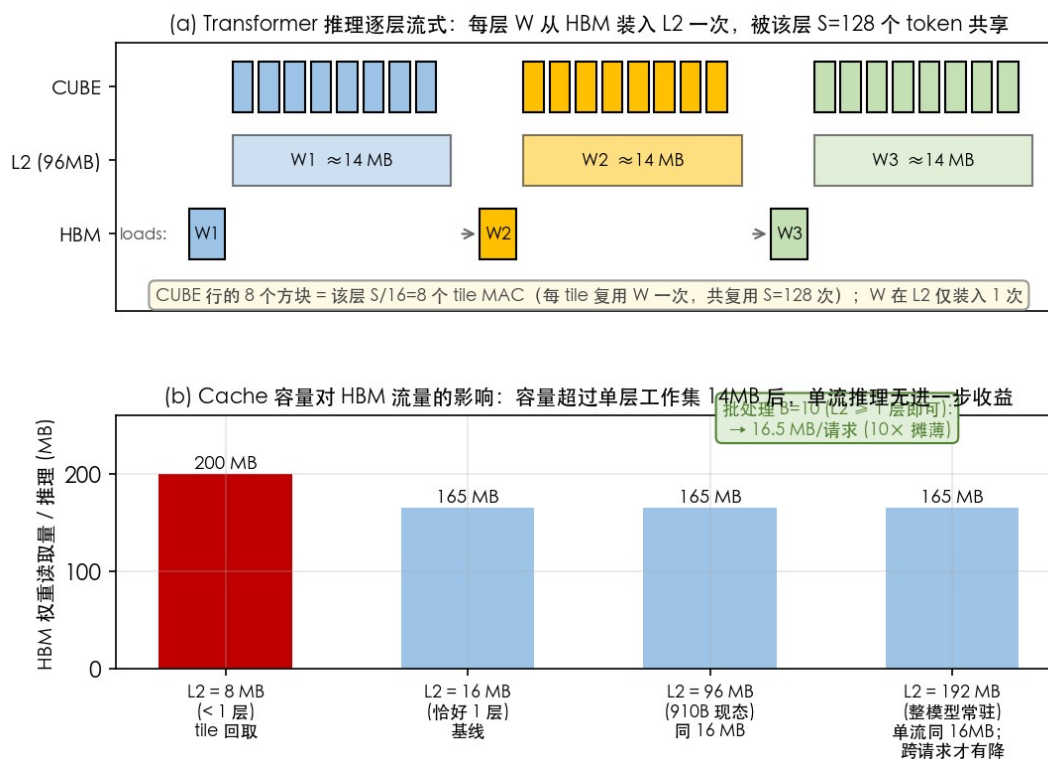


图12 (Q4) (a) 每层  $W$  从 HBM 装入 L2 一次，被该层  $S$  个 token 复用；(b) Cache 容量超过单层工作集后，对单流推理的增益递减；batched 后 HBM 流量被  $B$  进一步摊薄

一次推理的访存与复用结构可归结为四点：（1）**跨层无复用**：12 层串行执行，处理完 Layer- $n$  即不再需要其权重，转入 Layer- $(n+1)$ ；整次推理的 HBM 权重总读量  $\approx \Sigma$  各层权重  $\approx 165 \text{ MB}$ （与模型大小一致）。（2）**层内高复用**：单层内权重  $W$ （约 14MB）被该层所有  $S=128$  个 token 共享，即一次 HBM 装入对应  $S$  次（ $\times B$  次，若 batch= $B$ ）MAC 复用——实际等效命中率因此极高。（3）**Cache 容量阈值**：只要  $L2 \geq \max(W_{\text{layer}})$ （单层权重最大值，14MB），层内复用即被完全吸收；L2 在 14MB 之下时会出现层内 tile re-fetch（命中率明显下降）。（4）**跨请求复用需要更大容量**：单流推理不会回看上一层，但若 L2 能容下整模型（ $\geq 165\text{MB}$ ），则连续多次请求间权重可常驻、跨请求 HBM 流量趋零。

## Q4.2 Cache 容量增加是否降带宽？分阶段看

| L2 容量                      | 对单流推理 HBM 流量影响 | 机理                             |
|----------------------------|----------------|--------------------------------|
| < 14MB                     | 显著上升           | 单层 tile re-fetch；层内复用失败        |
| 14MB（恰好单层）                 | 进入“一次性流式”稳态    | 每层 $W$ 从 HBM 装入 1 次，层内复用充分     |
| 96MB（910B 现状）              | 与 14MB 相同      | 已远超单层；单流场景无额外增益                |
| $\geq 165\text{MB}$ （容整模型） | 跨请求复用时趋零       | 对单流单次推理仍同 14MB；价值在跨请求/batch 场景 |

一句话：Cache 提升的边际收益在“容纳单层”之前最大；之后取决于**跨请求复用**（多请求 / batch / KV-cache 等场景）。本工况 batch $\approx 10$  已能将 HBM 流量摊薄约 10 倍/请求。

## Q4.3 实测命中率与带宽利用率

以上模式被真机实测确认（详见图 5/图 6）：

1. **L2 读命中率**：FP16 74–96%（小 batch 因 cold start 略低），INT8 95–99.8%；
2. **外部 HBM 带宽利用率**  $\approx 0.4\%$ （远低于 392 GB/s 峰值）——复用被片上 L2 完整吸收；
3. **三级活跃带宽**：HBM 1.4 / L2→L1 (mte2) 2.7 / L1→L0 (mte1) 122 GB/s ——越靠近 CUBE 流量越高、命中越好；
4. **精度对命中率的反向影响**：位宽减半 → 单层工作集减半（14→7MB）→ 更易驻 L2 → 命中率上升。即低精度有助于缓解访存压力，而非加剧。

口径说明：以上 L2 命中率指 AIC 端口的 L2 读命中（per-op msprof），是解释 HBM 闲置的正确口径；系统级 llc\_read\_write 计数器另读约 46%，对应更窄的 LLC 结构，二者口径不同。

## Q4.4 对齐与 fractal 内维

910B 采用 fractal/NZ 格式、按 512B 对齐搬运；低精度通过加深 fractal 内维保持 512B 对齐：FP16 为 16×16、INT8 为 16×32、FP4 为 16×64。只要模型归约维（K）为 32/64 的整数倍即无 tile 边缘浪费——ET-BERT 的  $K \in \{64, 128, 768, 3072\}$  全部满足，FP4 下亦无对齐导致的额外开销。

四、下一代架构探索：低精度的收益边界

将上述实测外推到两项下一代改动：① 专用电路吸收量化/反量化开销（理想 fixpipe）；② 外部 HBM 带宽翻倍。投影以真机实测为锚（理想 fixpipe 通过对真机 per-op 分解、剥离 requant 算子得到）：

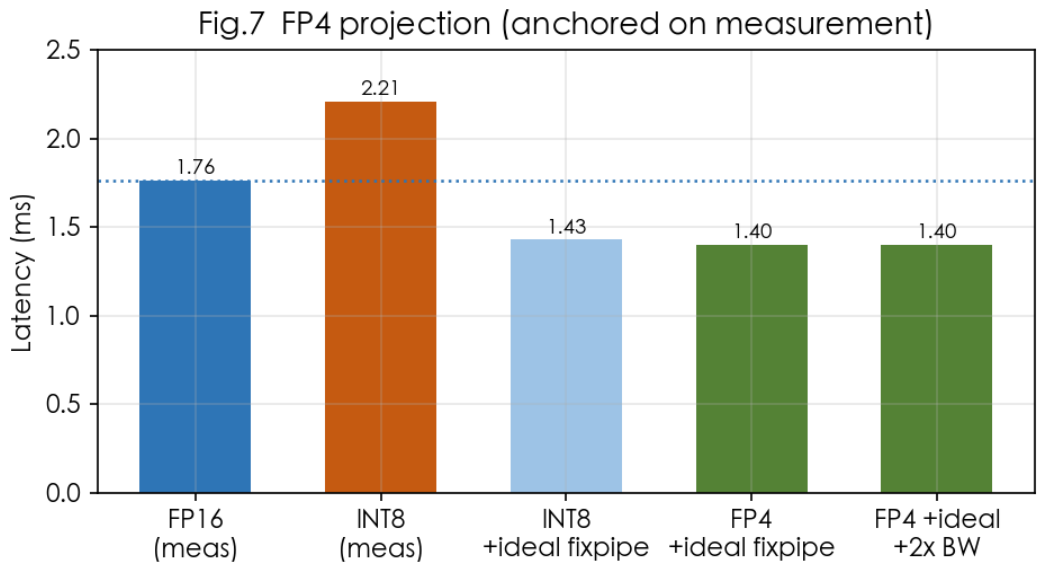


图7 FP4 端到端时延投影（以实测为锚）

| 场景                           | 时延 (ms) | 相对 FP16 | 说明                   |
|------------------------------|---------|---------|----------------------|
| FP16（实测）                     | 1.76    | 1.00×   | 基线                   |
| INT8（实测，含 requant）           | 2.21    | 1.26×   | requant 开销使其偏慢       |
| FP4 + 理想 fixpipe             | 1.40    | 0.80×   | 消除 requant 后转为净收益    |
| FP4 + 理想 fixpipe + 2× HBM 带宽 | 1.40    | 0.80×   | 外部 HBM 非当前限制因素，故增益有限 |

**读法（客观陈述）：**（1）专用低精度电路（消除 requant）可使 FP4 由偏慢转为约 20% 净收益，是有价值的方向；（2）在该工况下，提升外部 HBM 带宽对端到端时延的直接作用有限，因为外部 HBM 并非当前限制因素；（3）低精度的 4× 算力需在 host 与片上访存约束被同时缓解后才能更充分体现——这与第二节的瓶颈定位一致。

五、下一代架构建议

以下建议按“工况贴合度 + 投入产出”排序，均为客观陈述；提速倍数中标注为“实测”者来自真机，标注为“模型投影”者来自架构敏感度分析。

A. 运行时 / 编译层：降低 host 调度开销

该方向收益最大且属软件演进的必然路径；其量化收益如下图（架构模型投影）：

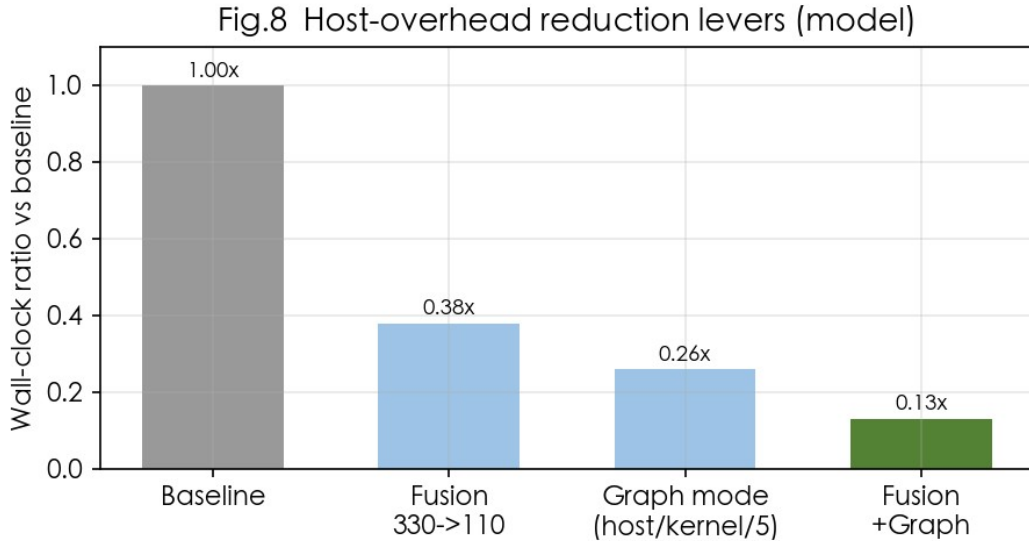


图8 host 开销优化的时延比（模型投影）：融合 + 整图下沉可达约 0.13×

5. **整图下沉 / 静态图执行**：减少逐 kernel 的 host 派发。实测依据：大 kernel 模型 host 开销约 8.4μs/核，而本工况小 kernel 高达 41.7μs/核，调度在小 kernel 上被完全暴露。
6. **算子融合**：FlashAttention、融合 QKV、融合 FFN+激活、融合 LayerNorm+残差，将 kernel 数由约 330 降至约 110。模型投影：kernel 数降至 1/3 → 时延约 0.38×；叠加整图下沉 → 约 0.13×。
7. **攒批 / 持续批处理**：摊薄固定开销。实测：固定开销占比由 b=1 的 88% 降至 b=64 的 10%，吞吐由 571 升至 4306 样本/秒。

## B. 访存路径：设备侧的持久核心杠杆

host 开销被优化后，设备侧关键路径落在 mte2（L1 填充）。对低 S、小 batch、小矩阵的加密流量类负载，该路径尤为关键：

8. **提升 L2→L1 填充带宽、增加 MTE2 引擎 / DMA 队列深度**：直接缩短设备侧关键路径。架构模型佐证：该路带宽翻倍 → AIC 时间约减半；而 L1↔L0（mte1）实测仅用约 6% 峰值带宽，提升 L1↔L0 带宽对本工况作用有限。
9. **增大 L1 / L2 容量并优化 tiling（weight-stationary，提高每次填充的复用行数）**：减少重复填充，提升命中。实测命中率已较高（L2 读 74–99%），增益主要体现在更大工作集与多请求场景。
10. **HBM 外部带宽**：当前工况下外部 HBM 利用率低，提升它对该类负载的直接收益有限；但当工作集增大、batch 升高或采用更大序列时，其重要性会上升，宜按目标工况权衡。

## C. 低精度：以减小数据足迹缓解访存

11. **专用低精度 fixpipe（吸收 requant）**：使低精度由（含量化开销时的）偏慢转为净收益，投影约 +20%（FP4），建议与 A、B 协同推进。
12. **低精度的访存价值**：位宽减半使权重/激活/KV 足迹减半，提升 cache 命中、降低 mte2 与外部访存压力——与 B 方向相互增益，是低精度在本工况下的主要收益来源。
13. **更高的低精度算力（如 FP4 4×）**：在 host 与片上访存约束缓解前难以充分体现；建议作为 A、B 落地后的进一步收益项，并配合 Q2 的带宽预算评估。

六、总结

| 问题           | 结论                              | 关键实测证据                                                             |
|--------------|---------------------------------|--------------------------------------------------------------------|
| Q1 公平算 MFU   | 各精度按各自峰值归一、按耗时加权；勿跨精度直接比数值      | CUBE 利用率 FP16 32→75%，INT8 14→46%                                   |
| Q2 FP4 带宽需求  | 搬运字节量减半，但维持峰值所需带宽翻倍             | FP8 74 → FP4 147 GB/s                                              |
| Q3 降精度代价     | MFU 降、需更大访存带宽；实测 INT8 偏慢 26%    | 1.76→2.21ms；requant 占 19–35%                                       |
| Q4 Cache 与访存 | 16MB 装不下整模型但无需；低精度命中更高、HBM 更闲   | L2 命中 FP16 74–96% / INT8 95–99.8%；HBM 0.4%                         |
| 瓶颈定位         | host 调度（可优化）+ 访存搬运（持久核心）；计算非瓶颈  | AIC 主导 pipe 恒为 mte2；S=128–512 × b=1/10 全工况 mac/mte2 ∈ [0.37, 0.89] |
| 下一代          | 专用低精度电路 + 访存路径增强 + host 运行时三者协同 | FP4 理想 1.40ms（约 +20%）                                              |

**核心判断：**对加密流量识别这类低序列、小 batch、小矩阵的负载，除 host 调度开销外，**访存（片上数据搬运）是设备侧的核心瓶颈**；host 开销随运行时演进将被优化，访存约束则是持久的架构杠杆。低精度的价值主要在于减小数据足迹、缓解访存，与访存路径增强相互增益；其更高的算力需在 host 与访存约束缓解后方能充分体现。

附·完整性说明：FP16/INT8 为 910B4 真机实测（ais\_bench + msprof + AMCT PTQ）；FP8/FP4 为外推（无对应硅）。“理想 fixpipe”由真机 per-op OP-Type 分解剥离 requant 算子得到，非纯建模。“整图下沉/算子融合”的提速倍数为架构敏感度模型投影，结合实测 kernel 数与 host 开销关系推算；端到端 FlashAttention/SDPA 融合真机实测因平台环境（transformers 5.7 与 ONNX 导出器不兼容）暂未完成，列为后续补测项。S=128/256/512 × batch=1/10 全工况 pipe 分解已 910B4 真机实测（与 ET-BERT b1 在 S=128 上 mac/mte2 逐位吻合，验证一致性）。